

8강. 성능 최적화 & 전역 관리



React

Hook – useRef()

▪ useRef() 훅

컴포넌트에서 값을 저장하거나 DOM 요소에 직접 접근할 때 사용하는 훅(Hook)입니다.

```
const ref = useRef(initialValue);
```

- ref.current에 값이 저장됨 - **{current: value}**
- 값이 변경되어도 컴포넌트가 다시 렌더링되지 않음

사용 예)

- 입력창 자동 포커스
- 이전 값 저장
- 타이머 ID 저장 (setTimeout, setInterval)
- 외부 라이브러리 DOM 제어



Hook – useRef()

- 값 저장(렌더링과 무관한 데이터)



렌더링 ...

▶ {current: 0}

렌더링 ...

▶ {current: 0}

렌더링 ...

▶ {current: 0}

Ref: 1

Ref: 2

Ref: 3

Var: 1

Var: 2

Var: 3



Hook – useRef()

- 값 저장(렌더링과 무관한 데이터) – Counter.jsx

```
function Counter() {  
  const [count, setCount] = useState(0);  
  // useRef 혹은 사용하여 countRef라는  
  // 참조 객체를 생성하고 초기값을 0으로 설정  
  const countRef = useRef(0);  
  // 일반 변수 - 컴포넌트가 리렌더링될 때마다 초기화  
  let countVar = 0;  
  
  console.log("렌더링...");  
  // {current: 0} 객체 - 초기값은 0입니다.  
  // countRef.current를 사용  
  console.log(countRef);  
  
  // 상태 증가  
  const increaseCount = () => {  
    |   setCount(count + 1);  
  }  
}
```



Hook – useRef()

- 값 저장(렌더링과 무관한 데이터)

```
// 참조 증가
const increaseCountRef = () => {
  countRef.current = countRef.current + 1;
  console.log("Ref:", countRef.current);
}

// 일반 변수 증가
const increaseCountVar = () => {
  countVar = countVar + 1;
  console.log("Var:", countVar);
}

return (
  <div>
    <p>State: {count}</p>
    <p>Ref: {countRef.current}</p>
    <p>Var: {countVar}</p>

    <button onClick={increaseCount}>State 증가</button>
    <button onClick={increaseCountRef}>Ref 증가</button>
    <button onClick={increaseCountVar}>Var 증가</button>
  </div>
);
```



Hook – useRef()

- DOM 요소 직접 접근

페이지가 로딩되면 포커스가 자동으로 위치하도록 함



A rectangular form containing a text input field on the left and a submit button on the right. The input field has a light gray border and contains the placeholder text "이름을 입력하세요" (Enter name). The submit button is a small rectangle with the text "전송" (Submit).

```
const ref = useRef(initialValue);
```

```
<input ref={ref} />
```



Hook – useRef()

- DOM 요소 직접 접근 – InputFocus.jsx

```
function InputFocus() {  
  // useRef 혹은 사용하여 input 요소에 대한 참조를 생성  
  const inputRef = useRef(null);  
  console.log(inputRef); // {current: null} 객체 - 초기값은 null입니다.  
  
  useEffect(() => {  
    console.log(inputRef);  
    // 컴포넌트가 마운트된 후 input 요소에 포커스를 설정  
    inputRef.current.focus();  
  }, []);  
}
```

```
▼ {current: input} ⓘ  
  ▼ current: input  
    value: (...)  
    ▶ __reactEvents$vm035nkh09r: Set(1) {  
    ▶ __reactFiber$vm035nkh09r: FiberNode
```



Hook – useRef()

▪ DOM 요소 직접 접근 – InputFocus.jsx

```
const handleSubmit = () => {  
  // input 요소의 현재 값을 알림으로 표시  
  alert(`환영합니다. ${inputRef.current.value}님`);  
  
  // 버튼 클릭 후에도 input 요소에 포커스 유지  
  inputRef.current.focus();  
  // 버튼 클릭 후 input 요소의 값을 초기화  
  inputRef.current.value = "";  
};  
  
return (  
  <div>  
    { /* ref 속성은 특정 DOM 요소에 접근할 수 있게 해줍니다. */}  
    <input  
      type="text"  
      ref={inputRef}  
      placeholder="이름을 입력하세요"  
    />  
    <button onClick={handleSubmit}>전 송</button>  
  </div>  
);
```

Ref 속성이 있어야
{current: input}



Hook – useCallback()

- 메모이제이션(Memoization) 이란?

한 번 계산(또는 생성)한 결과를 저장해두고, 다음에 같은 조건이면 다시 계산하지 않고 재사용하는 것

$1 + 1 = 2$

처음엔 계산

☞ 다음엔 그냥 "2"를 기억해서 바로 사용

```
const handleClick = () => {  
  console.log("클릭");  
};
```

렌더링될 때마다 새로운 함수 생성됨(주소가 바뀜)



Hook – useCallback()

- **useCallback()**

함수를 “재사용(캐싱)”해서 불필요한 재생성을 막는 리액트 훅입니다.

```
const memoizedFn = useCallback(() => {  
  실행코드;  
}, [item]);
```

렌더링 되어도 같은 함수 재사용(캐싱) – 초기화되지 않음

구조

- 첫번째 인자: 메모이제이션 해줄 콜백함수
- 두번째 인자: 의존성 배열



Hook – useCallback()

- useCallback()

number: 4

```
handleClick 함수가 변경되었습니다.  
handleClick 함수가 변경되었습니다.  
4 handleClick 함수가 변경되었습니다.  
number = 4  
handleClick 함수가 변경되었습니다.  
handleClick 함수가 변경되었습니다.
```



Hook – useCallback()

- useCallback()

```
import { useState, useCallback, useEffect } from "react";

// useCallback 혹은 사용하여 함수를 메모이제이션
function UseCallbackExample() {

  const [number, setNumber] = useState(0);
  /* 클릭하면 handleClick이 계속 변경되는 이유는 컴포넌트가 리렌더링될 때마다
  handleClick 함수(변수초기화)가 새로 생성되기 때문임.
  하지만 useCallback 혹은 사용하여 handleClick 함수를 메모이제이션하면,
  number가 변경될 때만 handleClick 함수가 새로 생성되고,
  그렇지 않을 때는 이전에 생성된 함수를 재사용하게 됩니다. */

  /*const handleClick = () => {
    console.log("handleClick 실행, number=", number);
  }*/
```



Hook – useCallback()

- useCallback()

```
useEffect(() => {  
  console.log("handleClick 함수가 변경되었습니다.");  
}, [handleClick]); // handleClick이 변경될 때마다 이 useEffect가 실행  
  
return (  
  <div>  
    <h3>number: {number}</h3>  
    <input  
      type="number"  
      value={number}  
      onChange={(e) => setNumber(Number(e.target.value))}</input>  
    <button onClick={handleClick}>클릭</button>  
  </div>  
>);  
}
```



Hook – useCallback()

- **useCallback()**

핵심 차이는 **함수의 재생성 여부**(참조 동일성) 입니다.

function handleClick() { ... } 또는 const handleClick = () => { ... }

컴포넌트가 리렌더링될 때마다 새 함수가 만들어집니다.

즉, 이전 렌더의 함수와 현재 함수는 **===** 비교 시 보통 다릅니다.

보통은 이걸로 충분하고, 성능 문제 없으면 가장 단순합니다!

const handleClick = useCallback(() => { ... }, [deps])

deps가 바뀌지 않으면 같은 함수 참조를 재사용합니다.

deps가 바뀌면 그때 새 함수가 만들어집니다.



상태 관리 – useReducer()

▪ useReducer() 함수

리액트의 useReducer는 상태(state)를 더 체계적으로 관리하기 위한 훅입니다. 특히 상태 변화 로직이 복잡할 때 useState보다 훨씬 유용합니다. 즉, 상태 변경 로직을 함수로 분리해서 관리하는 상태 관리 훅입니다.

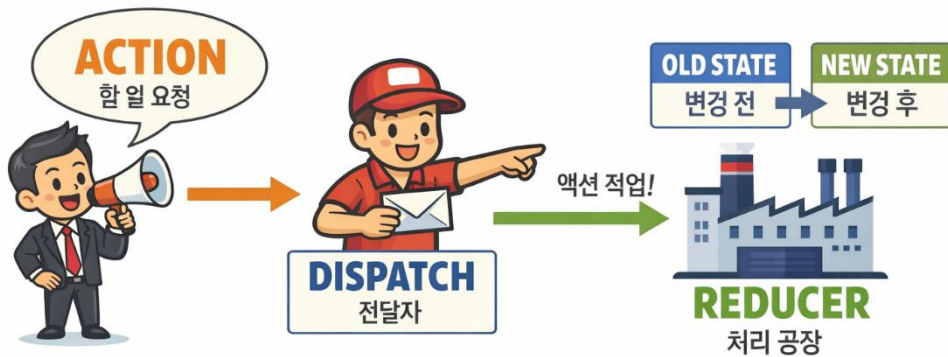
```
const [state, dispatch] = useReducer(reducer, initialState);
```

구성 요소

- **state** → 현재 상태
- **dispatch** → 상태를 변경하기 위해 action을 전달하는 함수
- **reducer** → 현재 상태와 액션을 받아 새로운 상태를 반환하는 함수
- **initialState** → 초기 상태



상태 관리 – useReducer()



▪ reducer() 함수 구조

```
const reducer = (state, action) => {  
  ;  
};
```

- 반드시 이전 state를 기반으로 새로운 state 반환



상태 관리 – useReducer()

▪ useState vs useReducer

구분	useState	useReducer
사용 난이도	쉬움	조금 어려움
상태 구조	단순	복잡
로직 관리	컴포넌트 내부	reducer로 분리
유지보수	보통	매우 좋음

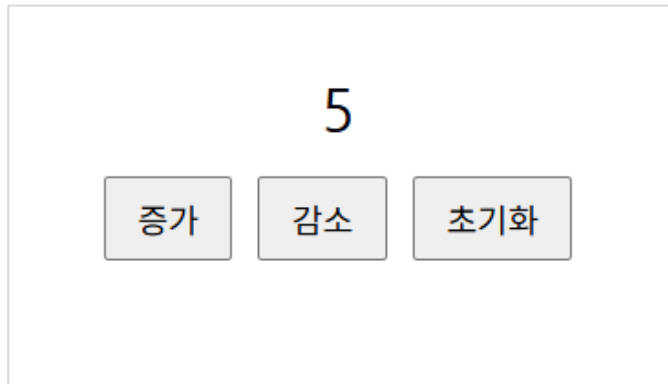
useReducer가 좋은 상황

- 상태가 객체/배열로 복잡할 때
- 상태 변경 로직이 여러 곳에서 반복될 때
- 조건 분기가 많을 때 (if, switch)
- 폼, 장바구니, 인증 상태 관리 등



상태 관리 – useReducer()

■ useReducer 사용 예제



```
▼ {count: 0} ⓘ ▶ {type: 'INCREMENT'}  
  count: 0  
  ▶ [[Prototype]]: Object  
▼ {count: 1} ⓘ ▼ {type: 'INCREMENT'} ⓘ  
  count: 1      type: "INCREMENT"  
  ▶ [[Prototype]]: Object ▶ [[Prototype]]: Object  
▶ {count: 2} ▶ {type: 'INCREMENT'}  
▶ {count: 3} ▶ {type: 'INCREMENT'}
```



상태 관리 – useReducer()

▪ useReducer 사용 예제 – CountReducer.jsx

```
import React, { useReducer } from "react";

/* reducer 함수는 현재 상태와 액션을 받아
   | 새로운 상태를 반환하는 함수입니다. */
const reducer = (state, action) => {
  console.log(state, action);

  // 액션의 타입에 따라 상태를 업데이트하는 로직을 구현합니다.
  switch (action.type) {
    case "INCREMENT":
      return { count: state.count + 1 };
    case "DECREMENT":
      return { count: state.count - 1 };
    case "RESET":
      return {count: 0};
    default:
      return state;
  }
};
```



상태 관리 – useReducer()

▪ useReducer 사용 예제 – CountReducer.jsx

```
const CounterReducer = () => {  
  // useReducer 혹은 사용하여 상태와 디스패치 함수를 생성합니다.  
  // initialState는 { count: 0 }으로 설정되어 있습니다.  
  const [state, dispatch] = useReducer(reducer, {count: 0});  
  
  return (  
    <div>  
      <h2>{state.count}</h2>  
      <button onClick={() =>  
        dispatch({ type: "INCREMENT" })}>증가</button>  
      <button onClick={() =>  
        dispatch({ type: "DECREMENT" })}>감소</button>  
      <button onClick={() =>  
        dispatch({ type: "RESET" })}>초기화</button>  
    </div>  
  );  
};
```



상태 관리 – useReducer()

- 은행 입, 출금 거래

현재 잔액: 2000원

예금

출금



상태 관리 – useReducer()

▪ 은행 입, 출금 거래 – BankReducer.jsx

```
const initialState = {  
  | balance: 0 // 초기 잔액을 0으로 설정합니다.  
}  
  
const bankReducer = (state, action) => {  
  console.log("reducer가 작동합니다.", state, action);  
  
  switch (action.type) {  
    // 예금 시에는 현재 잔액에 액션의 payload를 더하여 새로운 상태를 반환합니다.  
    case 'DEPOSIT':  
      return { ...state, balance: state.balance + action.payload };  
    // 출금 시에는 현재 잔액에서 액션의 payload를 빼서 새로운 상태를 반환합니다.  
    case 'WITHDRAW':  
      if (state.balance < action.payload) {  
        alert("잔액이 부족합니다.");  
        return state; // 잔액이 부족할 경우 상태를 변경하지 않고 그대로 반환합니다.  
      }  
      return { ...state, balance: state.balance - action.payload };  
    default:  
      return state; // 현재 상태를 그대로 반환합니다.  
  }  
};
```



상태 관리 – useReducer()

▪ 은행 입, 출금 거래 – BankReducer.jsx

```
const BankReducer = () => {  
  const [amount, setAmount] = useState(0);  
  const [state, dispatch] = useReducer(bankReducer, initialState);  
  
  return (  
    <div>  
      <h3>현재 잔액: {state.balance}원</h3>  
      <input  
        type="number"  
        value={amount}  
        onChange={(e) => setAmount(parseInt(e.target.value))}  
        step="1000"  
      />  
      <button onClick={() =>  
        dispatch({ type: 'DEPOSIT', payload: amount })}>예금</button>  
      <button onClick={() =>  
        dispatch({ type: 'WITHDRAW', payload: amount })}>출금</button>  
    </div>  
  )  
}
```



전역 공유 – useContext()

- useContext()

리액트의 `useContext`는 컴포넌트 트리 전체에서 데이터를 쉽게 공유하기 위한 hook입니다.

쉽게 말하면 **props 없이** 데이터 전달하는 방법입니다.

✓ props drilling 문제 - 계속 전달해야 함

```
<Parent user={user} />  
  <Child user={user} />  
    <GrandChild user={user} />
```



전역 공유 – useContext()

▪ useContext() 3단계

1. Context 생성

```
import { createContext } from "react";  
  
export const UserContext = createContext();
```

2. Provider로 감싸기

```
<UserContext.Provider value={user}>  
  <App />  
</UserContext.Provider>
```

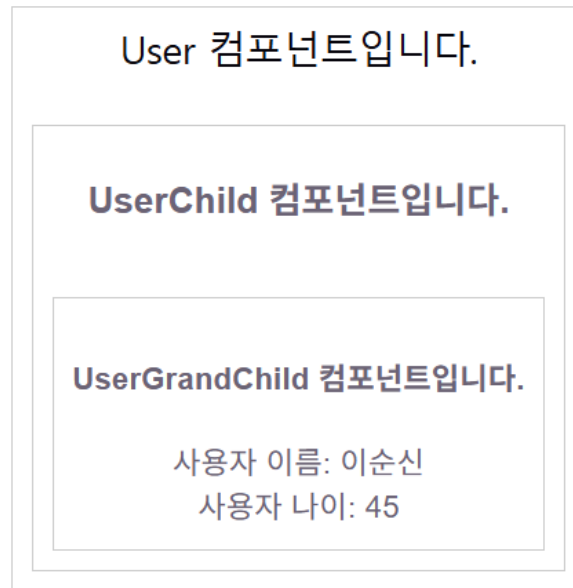
3. 값 사용

```
const Profile = () => {  
  const user = useContext(UserContext);  
  
  return <h2>{user.name}</h2>;  
};
```



전역 공유 – useContext()

▪ useContext() 사용 예제 – user 객체 전달



User → Provider로 user 데이터 제공
└ UserChild → user 데이터 몰라도 됨 (단순 중간 컴포넌트)
└ UserGrandChild → useContext로 직접 user 데이터 접근



전역 공유 – useContext()

- useContext() – User.jsx

```
import { createContext, useContext } from 'react';
import UserChild from './UserChild';

// useContext를 생성합니다. 기본값은 빈 객체입니다.
export const UserContext = createContext({});

const User = () => {
  // user 데이터를 정의합니다.
  const user = {
    name: "이순신",
    age: 45,
  };
};
```



전역 공유 – useContext()

- useContext() – User.jsx

```
return (  
  <div className='user'>  
    <h2>User 컴포넌트입니다.</h2>  
    { /* useContext.Provider로 감싸서 user 값을  
      | | | | | 하위 컴포넌트에 제공합니다 */ }  
    <UserContext.Provider value={user}>  
      <UserChild />  
    </UserContext.Provider>  
  </div>  
);  
}  
  
export default User;
```



전역 공유 – useContext()

- useContext() – UserChild.jsx

```
import { useContext } from 'react';
import UserGrandChild from './UserGrandChild';

const UserChild = () => {

  return (
    <div className='user-child'>
      <h3>UserChild 컴포넌트입니다.</h3>
      <UserGrandChild />
    </div>
  );
}

export default UserChild;
```



전역 공유 – useContext()

- useContext() – UserGrandChild.jsx

```
import { useContext } from 'react';
import { UserContext } from '../User';

const UserGrandChild = () => {
  // UserContext에서 user 데이터를 가져옵니다.
  const user = useContext(UserContext);

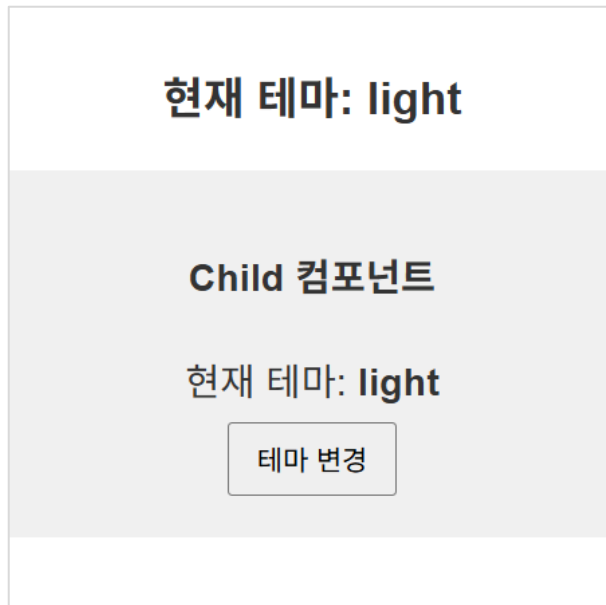
  return (
    <div className='user-grandchild'>
      <h4>UserGrandChild 컴포넌트입니다.</h4>
      <p>사용자 이름: {user.name}</p>
      <p>사용자 나이: {user.age}</p>
    </div>
  );
}

export default UserGrandChild;
```



전역 공유 – useContext()

- useContext() 사용 예제 – 테마 변경



전역 공유 – useContext()

- useContext() 사용 예제 – ParentTheme.jsx

```
import { createContext, useState } from "react"
import ChildTheme from "../ChildTheme";

// Context 생성
export const ThemeContext = createContext("light");

const ParentTheme = () => {
  const [currentTheme, setCurrentTheme] = useState("light");

  // 테마 토글 함수
  const toggleTheme = () => {
    setCurrentTheme(prevTheme => (
      prevTheme === 'light' ? 'dark' : 'light'
    ))
  }
}
```



전역 공유 – useContext()

▪ useContext() 사용 예제 – ParentTheme.jsx

```
// 부모 컴포넌트 스타일
const parentStyle = {
  background: currentTheme === 'dark' ? '■#333' : '□#fff',
  color: currentTheme === 'dark' ? '□#fff' : '■#333',
  padding: '20px'
}

// 부모 컴포넌트 렌더링
return(
  <div style={parentStyle}>
    <h3>현재 테마: {currentTheme}</h3>
    {/* 하위 컴포넌트에 Context 제공 */}
    <ThemeContext.Provider value={{currentTheme, toggleTheme}}>
      <ChildTheme />
    </ThemeContext.Provider>
  </div>
)
```



전역 공유 – useContext()

▪ useContext() 사용 예제 – ChildTheme.jsx

```
import { useContext } from "react"
import { ThemeContext } from "../ParentTheme"

const ChildTheme = () => {
  // Context 사용
  const { currentTheme, toggleTheme } = useContext(ThemeContext);
  console.log("현재 테마:", currentTheme);

  // 자식 컴포넌트 스타일
  const childStyle = {
    background: currentTheme === 'dark' ? '■ #555' : '□ #f0f0f0',
    color: currentTheme === 'dark' ? '□ #fff' : '■ #333',
    padding: '15px',
  };
};
```



전역 공유 – useContext()

▪ useContext() 사용 예제 – ChildTheme.jsx

```
// 자식 컴포넌트 렌더링
return(
  <div style={childStyle}>
    <h4>Child 컴포넌트</h4>
    <p>현재 테마: <strong>{currentTheme}</strong></p>
    <button onClick={toggleTheme}>
      |   테마 변경
    </button>
  </div>
)
}

export default ChildTheme
```



리덕스(Redux)

▪ Redux란?

애플리케이션의 전역 상태(state)를 관리하기 위한 상태 관리 라이브러리입니다. 규모가 커질수록 복잡해지는 상태 관리를 예측 가능하게 만들어주는 것이 핵심입니다.

▪ 기존 리엑트 애플리케이션의 단점

- 탭-다운 방식을 사용하므로 부모-자식 사이에서만 데이터를 전달할 수 있다.
- 자식 컴포넌트 사이에서는 직접적인 데이터 전달이 불가능하다.
- 컴포넌트가 많아질수록 상태 관리는 복잡하고 어려워진다.



리덕스(Redux)

▪ Redux와 useReducer의 차이

구분	useReducer	Redux
범위	컴포넌트 내부	전역
설치	필요 없음	필요 (<code>redux</code> , <code>react-redux</code>)
복잡도	낮음	높음
상태 공유	어렵다 (props/context 필요)	매우 쉬움
사용 목적	로컬 상태 관리	앱 전체 상태 관리
구조	간단	구조화된 아키텍처



리덕스(Redux)

▪ Redux란?

1) Store (저장소)

- 애플리케이션의 **모든 상태를 저장**

```
const store = createStore(reducer);
```

2) Action (행동)

- 상태를 변경하기 위한 **요청 객체**

```
{  
  type: "INCREMENT"  
}
```



리덕스(Redux)

▪ Redux란?

3) Reducer (상태 변경 함수)

- 이전 상태 + action → 새로운 상태 반환

```
function counterReducer(state = { count: 0 }, action)
{
  switch (action.type) {
    case "INCREMENT":
      return { count: state.count + 1 };
    case "DECREMENT":
      return { count: state.count - 1 };
    default:
      return state;
  }
}
```



리덕스 툴킷(Redux Toolkit)

- **Redux의 단점**

액션 타입정의, 액션 생성자, 리듀서 작성 등 많은 설정과 반복적인 코드가 필요하다.

- **Redux 툴킷(Tool-Kit)**

Redux Toolkit(RTK)은 **Redux**를 더 쉽고 간결하게 사용하기 위한 공식 라이브러리입니다.

리덕스의 핵심 개념은 유지하면서도 createSlice, configureStore 같은 API를 제공하여 복잡한 설정을 줄이고, 간결하면서도 효율적으로 상태관리를 구현할 수 있도록 도와줍니다.



리덕스 툴킷(Redux Toolkit)

- React Redux 공식 사이트

 **React Redux**

Getting Started Tutorial

Introduction

Getting Started

Why Use React Redux?

Tutorials

Quick Start

TypeScript Quick Start

Tutorial: Connect API

Using React Redux

Usage with TypeScript

Connect: Extracting Data with mapStateToProps

Connect: Dispatching Actions with mapDispatchToProps

Accessing the Store

API Reference

Guides

Usage Summary

Install Redux Toolkit and React Redux

Add the Redux Toolkit and React Redux packages to your project:

```
npm install @reduxjs/toolkit react-redux
```

Create a Redux Store

Create a file named `src/app/store.js`. Import the `configureStore` from `@reduxjs/toolkit`, creating an empty Redux store, and exporting it:

```
app/store.js

import { configureStore } from '@reduxjs/toolkit'

export default configureStore({
  reducer: {},
})
```

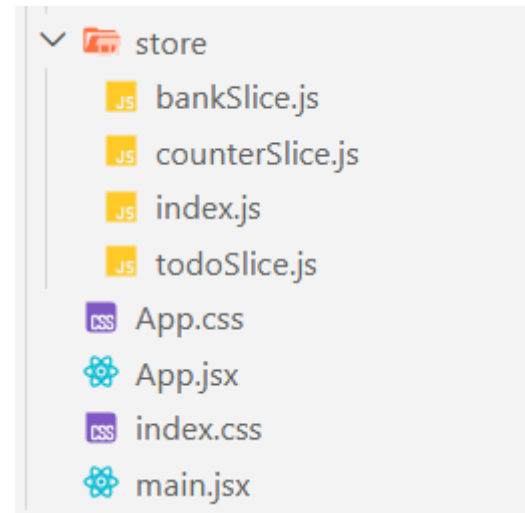


리덕스 툴킷(Redux Toolkit)

- Redux 툴킷(Tool-Kit) 설치

```
npm install @reduxjs/toolkit react-redux
```

```
"dependencies": {  
  "@reduxjs/toolkit": "^2.11.2",  
  "react": "^19.2.4",  
  "react-dom": "^19.2.4",  
  "react-redux": "^9.2.0",  
  "react-router-dom": "^7.14.0"  
},
```



리덕스 툴킷(Redux Toolkit)

- **createSlice** - action + reducer를 한 번에 생성

```
import { createSlice } from '@reduxjs/toolkit'
const initialState = {
  count: 0 // 카운트의 초기값을 0으로 설정
}

const counterSlice = createSlice({
  name: 'counter', // slice의 이름을 지정
  initialState,
  reducers: {
    // 카운트 값을 1 증가시키는 reducer
    increment: (state) => {
      state.count += 1
    },
  }
})
// 액션 생성자와 리듀서를 export
export const { increment } = counterSlice.actions
export default counterSlice.reducer
```

counterSlice.js



리덕스 툴킷(Redux Toolkit)

- store 생성 – configureStore()

reducer를 등록

index.js

```
import { configureStore } from '@reduxjs/toolkit'
import counterReducer from './counterSlice'

export const store = configureStore({
  reducer: {
    // counterSlice에서 export한 reducer를 등록
    counter: counterReducer,
  }
})
```



리덕스 툴킷(Redux Toolkit)

- **Provider**

Redux의 store를 React와 연결하기 위해 사용

main.jsx

```
import { Provider } from 'react-redux'
import { store } from './store/index.js'

createRoot(document.getElementById('root')).render(
  <Provider store={store}>
    <App />
  </Provider>
)
```



리덕스 툴킷(Redux Toolkit)

- **useSelector**

store의 state를 가져오고, state가 변할 때마다 업데이트 함

```
import { useSelector, useDispatch } from "react-redux"
// counterSlice에서 export한 액션 생성 함수를 가져옴
```

apps/counter.jsx

```
const Counter = () => {
  // store의 state에서 count 값을 가져옴
  const count = useSelector(state => state.counter.count)
```



리덕스 툴킷(Redux Toolkit)

- **useDispatch**

store의 dispatch 메서드를 반환, action을 dispatch 할 수 있음

```
// dispatch 함수를 가져옴 - 액션을 dispatch할 때 사용
```

```
const dispatch = useDispatch()
```

apps/counter.jsx

```
return (
```

```
  <div className="redux-app">
```

```
    <h2>Counter</h2>
```

```
    <div>
```

```
      <p>Count: {count}</p>
```

```
      <button onClick={() => dispatch(increment())}>증가</button>
```



Counter 앱

- Counter

Counter

Bank

Todos

Counter

Count: 0

증가

감소

초기화

입력값만큼 증가



Counter 앱

▪ Counter

```
function App() {  
  return (  
    <>  
      <section id="center">  
        <BrowserRouter>  
          <Header />  
          <Routes>  
            <Route path="/" element={<Counter />} />  
            <Route path="/counter" element={<Counter />} />  
            <Route path="/bank" element={<Bank />} />  
            <Route path="/todos" element={<Todos />} />  
          </Routes>  
        </BrowserRouter>  
      </section>  
    </>  
  )  
}
```



Counter

- Counter

```
import { NavLink, Link } from "react-router-dom"

const Header = () => {
  return (
    <header>
      <nav>
        <NavLink to="/counter">Counter</NavLink>
        <NavLink to="/bank">Bank</NavLink>
        <NavLink to="/todos">Todos</NavLink>
      </nav>
    </header>
  )
}
```



Counter 앱

▪ Counter

```
import { configureStore } from '@reduxjs/toolkit'
import counterReducer from './counterSlice'
import bankReducer from './bankSlice'
import todoReducer from './todoSlice'

export const store = configureStore({
  reducer: {
    // counterSlice에서 export한 reducer를 등록
    counter: counterReducer,

    // bankSlice에서 export한 reducer를 등록
    bank: bankReducer,

    // todoSlice에서 export한 reducer를 등록
    todo: todoReducer
  }
})
```



Counter 앱

▪ Counter

```
import { createSlice } from '@reduxjs/toolkit'

const initialState = {
  count: 0 // 카운트의 초기값을 0으로 설정
}

const counterSlice = createSlice({
  name: 'counter', // slice의 이름을 지정
  initialState,
  reducers: { // state를 변경하는 reducer 함수를 정의
    // 카운트 값을 1 증가시키는 reducer
    increment: (state) => {
      state.count += 1
    },
    // 카운트 값을 1 감소시키는 reducer
    decrement: (state) => {
      state.count -= 1
    },
  },
})
```



Counter 앱

▪ Counter

```
// 카운트 값을 action.payload로 증가시키는 reducer
incrementByAmount: (state, action) => {
  state.count += action.payload
},
// 카운트 초기화
reset: (state) => {
  state.count = 0
}
})

// 액션 생성자와 리듀서를 export
export const {
  increment,
  decrement,
  incrementByAmount,
  reset
} = counterSlice.actions
export default counterSlice.reducer
```



Counter 앱

▪ Counter

```
import {
  increment,
  decrement,
  incrementByAmount,
  reset
} from "../store/counterSlice"
import { useState } from "react"

import { useSelector, useDispatch } from "react-redux"
// counterSlice에서 export한 액션 생성 함수를 가져옴

const Counter = () => {
  // store의 state에서 count 값을 가져옴
  const count = useSelector(state => state.counter.count)
  // input에 입력된 값을 저장할 state
  const [inputValue, setInputValue] = useState(0)

  // input 값이 변경될 때마다 state를 업데이트하는 함수
  const handleInputChange = (e) => {
    setInputValue(Number(e.target.value))
  }
}
```



Counter 앱

▪ Counter

```
// dispatch 함수를 가져옴 - 액션을 dispatch할 때 사용
const dispatch = useDispatch()

// 입력된 값만큼 카운트를 증가시키는 함수
const handleIncrementAmount = () => {
  dispatch(incrementByAmount(inputValue))
}

return (
  <div className="redux-app">
    <h2>Counter</h2>
    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch(increment())}>증가</button>
      <button onClick={() => dispatch(decrement())}>감소</button>
      <button onClick={() => dispatch(reset())}>초기화</button>
    </div>
  </div>
)
```



Counter 앱

▪ Counter

```
    <div>
      <input
        type="number"
        onChange={handleInputChange}
      />
      <button onClick={handleIncrementAmount}>입력값만큼 증가</button>
    </div>
  </div>
)
}

export default Counter
```



Bank System 앱

- Bank

<u>Counter</u>	<u>Bank</u>	<u>Todos</u>
Bank System		
잔액: 2000원		
3000	입금	출금
거래 내역		
[2026. 4. 6. 오전 9:29:16]입금: 5000원		
[2026. 4. 6. 오전 9:29:20]출금: 3000원		



Bank System 앱

▪ Bank

```
import { createSlice } from '@reduxjs/toolkit'

const initialState = {
  balance: 0, // 은행 잔액을 저장하는 state
  transactions: [] // 거래 내역을 저장하는 배열
}

const bankSlice = createSlice({
  name: 'bank',
  initialState,
  reducers: {
    deposit: (state, action) => {
      state.balance += action.payload // 입금액을 잔액에 더함
      // 거래 내역에 입금 기록을 추가
      // push()를 사용하여 transactions 배열에 새로운 거래 객체를 추가
      state.transactions.push({
        type: 'deposit',
        amount: action.payload,
        // timestamp는 거래가 발생한 시간을 문자열로 저장
        timestamp: new Date().toLocaleString('ko-KR')
      })
    },
  },
})
```



Bank System 앱

▪ Bank

```
withdraw: (state, action) => {  
  if (state.balance >= action.payload) {  
    state.balance -= action.payload // 출금액을 잔액에서 뺌  
    // 거래 내역에 출금 기록을 추가  
    state.transactions.push({  
      type: 'withdraw',  
      amount: action.payload,  
      timestamp: new Date().toLocaleString('ko-KR')  
    })  
  } else {  
    alert("잔액이 부족합니다.")  
  }  
}  
})  
  
export const { deposit, withdraw } = bankSlice.actions  
export default bankSlice.reducer
```



Bank System 앱

■ Bank

```
import { useSelector, useDispatch } from "react-redux"
import { deposit, withdraw } from "../store/bankSlice"
import { useState } from "react"

const ReduxBank = () => {
  // 입금/출금할 금액을 관리하는 state
  const [amount, setAmount] = useState(0)
  // bankSlice의 balance 값을 가져옴
  const balance = useSelector(state => state.bank.balance)
  // bankSlice의 transactions 값을 가져옴
  const transactions = useSelector(state => state.bank.transactions)

  const dispatch = useDispatch(); // dispatch 함수를 가져옴

  // 입금 버튼 클릭 시 deposit 액션을 dispatch
  const handleDeposit = () => {
    dispatch(deposit(Number(amount)))
  }

  // 출금 버튼 클릭 시 withdraw 액션을 dispatch
  const handleWithdraw = () => {
    dispatch(withdraw(Number(amount)))
  }
}
```



Bank System 앱

▪ Bank

```
return (  
  <div>  
    <h2>Bank System</h2>  
    <h3>잔액: {balance}원</h3>  
    <input  
      type="number"  
      value={amount}  
      onChange={(e) => setAmount(e.target.value)}  
      step="1000"  
    />  
    <button onClick={handleDeposit}>입금</button>  
    <button onClick={handleWithdraw}>출금</button>  
    <h3>거래 내역</h3>  
    <ul>  
      {transactions.map((transaction, index) => (  
        <li key={index}>  
          [{transaction.timestamp}]  
          {transaction.type === 'deposit' ? '입금' : '출금'}: {transaction.amount}원  
        </li>  
      ))}  
    </ul>  
  </div>  
)
```



To-do 앱

- To-do app

Counter Bank Todos

할 일 관리

☐ 운동 하기

☒ ~~영화 보기~~

☐ 코딩 하기



To-do 앱

- To-do app

```
const initialState = {  
  |   todos: []  
  |  
  | }  
  
// createSlice 함수를 사용하여 todoSlice 생성  
const todoSlice = createSlice({  
  |   name: 'todo', // slice의 이름  
  |   initialState,  
  |   reducers: {  
  |     | // 새로운 할 일을 추가하는 reducer  
  |     | addTodo: (state, action) => {  
  |     |   | state.todos.push(action.payload)  
  |     |   | }  
  |   },  
  |  
  | })
```



To-do 앱

▪ To-do app

```
//할 일 체크 기능 추가
toggleTodo: (state, action) => {
  const todo = state.todos.find(todo => todo.id === action.payload)
  if (todo) {
    todo.completed = !todo.completed
  }
},
// 특정 id를 가진 할 일을 제거하는 reducer
removeTodo: (state, action) => {
  state.todos = state.todos.filter(todo => todo.id !== action.payload)
}
})

export const { addTodo, toggleTodo, removeTodo } = todoSlice.actions
export default todoSlice.reducer
```



To-do 앱

- To-do app

```
import { use, useState } from 'react'
import { useSelector, useDispatch } from 'react-redux'
import { addTodo, toggleTodo, removeTodo } from '../store/todoSlice'

const Todos = () => {
  const [inputValue, setInputValue] = useState('')
  const todos = useSelector(state => state.todo.todos)
  const dispatch = useDispatch()

  console.log(todos);

  // 할 일 입력 핸들러
  const handleInputChange = (e) => {
    // console.log(e.target.value);
    setInputValue(e.target.value)
  }
}
```



To-do 앱

▪ To-do app

```
// 할 일 추가 핸들러
const handleAddTodo = () => {
  if (inputValue.trim() !== '') {
    dispatch(addTodo({
      id: Date.now(), // 고유한 id 생성
      text: inputValue,
      completed: false
    })))
    setInputValue('')
  }
}

// 할 일 체크 기능 추가
const handleToggleTodo = (id) => {
  dispatch(toggleTodo(id))
}

// 할 일 삭제 기능 추가
const handleRemoveTodo = (id) => {
  dispatch(removeTodo(id))
}
```



To-do 앱

▪ To-do app

```
return (  
  <div>  
    <h2>할 일 관리</h2>  
    <input  
      type="text"  
      placeholder="할 일을 입력하세요"  
      value={inputValue}  
      onChange={handleInputChange}  
    />  
    <button onClick={handleAddTodo}>추가</button>  
    { /* 할 일 목록 */ }  
    <ul>  
      { todos.map(todo => (  
        <li key={todo.id} className={todo.completed ? 'completed' : ''}>  
          <input  
            type="checkbox"  
            checked={todo.completed}  
            onChange={() => handleToggleTodo(todo.id)}  
          />  
          {todo.text}  
          <button onClick={() => handleRemoveTodo(todo.id)}>삭제</button>  
        </li>  
      )) }  
    </ul>  
  </div>  
)
```

